

瀏覽器擴充套件實作計畫 (Browser Extension Plan)

背景與目標

將現有的淨讀者 PWA 轉換為 Chrome 擴充套件 (Manifest V3)。

主要目標：利用擴充套件 `Background Service Worker` 的特權，徹底繞過瀏覽器的 CORS 限制與 Cloudflare 的初步阻擋，實現直接且快速的網頁抓取。

架構設計

使用者在瀏覽器閱讀小說時，點擊擴充套件圖示 (Action)，擴充套件會開啟一個專屬的閱讀頁面 (類似現有 PWA)，並透過 Background Script 代理抓取目標網址的內容。

專案結構 (預計建立於 `publishHTML/pureReader-extension/`)

1. `manifest.json`

擴充套件設定檔，宣告 V3 標準與必要權限。

- `permissions`: `["storage", "activeTab"]`
- `host_permissions`: `["<all_urls>"]` (關鍵：賦予 Background Script 跨域存取所有網站的權限)
- `background`: 指向 `background.js`
- `action`: 設定點擊圖示的行為 (開啟閱讀器)

2. `background.js` (Service Worker)

核心代理層。負責監聽來自閱讀器頁面 (UI) 的訊息。

當收到 `{ type: 'FETCH_URL', url: '...' }` 時，直接使用原生 `fetch()` 獲取 HTML 並回傳給 UI。由於擁有 `host_permissions`，此請求沒有 CORS 限制。

3. `reader.html` & `reader.js` (閱讀器介面)

由目前的 `pureReader/index.html` 移植而來。

主要修改點：

- 代理策略替換：**移除原有的 Vercel/Cloudflare/CodeTabs 代理策略。將

`_buildProxyStrategies` 改寫為單一策略：透過 `chrome.runtime.sendMessage` 向 `background.js` 請求 HTML 內容。

- **儲存機制**：可選擇將 `localStorage` 升級為 `chrome.storage.local`，以獲得更好的跨裝置同步與擴充性（或暫時保留 `localStorage` 亦可運作）。

執行步驟

第一階段：基礎擴充套件建置

- ☐ 建立 `pureReader-extension/` 目錄與 `manifest.json`。
- ☐ 撰寫 `background.js` 實作跨域 Fetch 的訊息監聽器。
- ☐ 撰寫 `popup.js` (或 Action 腳本)，讓使用者點擊擴充套件圖示時，擷取當前分頁網址，並在新分頁開啟 `reader.html?url=...`。

第二階段：核心邏輯移植與改寫

- ☐ 複製 `pureReader/index.html` 至擴充套件目錄作為 `reader.html`。
- ☐ 修改 HTML 內的 JS：移除舊代理機制，接上 `chrome.runtime.sendMessage` 呼叫 background fetch。
- ☐ 測試 Jina Reader Fallback：若網站極度嚴格連真實瀏覽器的 Fetch 都擋，保留 Jina Reader 作為最後防線。

第三階段：測試與封裝

- ☐ 在 Chrome 瀏覽器中載入未封裝的擴充套件進行本地測試。
- ☐ 針對 `novel543.com` 等受 CF 保護網站進行抓取測試。

Open Questions

Note

發佈方式：擴充套件完成後，您可以選擇在本地「載入未封裝項目」直接自用，或者我也可以協助您打包成 ZIP 檔，讓您上架到 Chrome Web Store（需支付給 Google 一次性 5 美元開發者註冊費）。目前我們預設先以「本地自用」模式開發。

Verification Plan

1. 在 Chrome 擴充套件管理頁面載入 `pureReader-extension` 目錄。
2. 開啟 `https://look.twword.com/030346010/8001_320.html`。
3. 點擊擴充套件圖示，確認能成功開啟淨讀器和瞬間載入內容（不需等待 Proxy Timeout）。
4. 測試連續翻頁與預載功能是否正常。